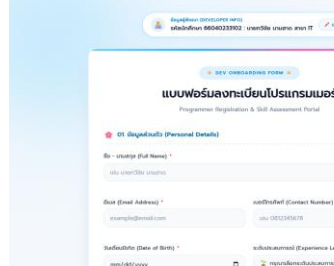


WorkShop 2 “การไหลของข้อมูล (ตัวแปร) ในโปรแกรม :DATA FLOW DETECTIVE WORKSHOP

รหัสนักศึกษา	ชื่อนักศึกษา	รูปภาพนักศึกษา
66040233102	นายทวิชัย นามฮาด	

รูปแบบงาน: งานเดี่ยว (Individual Work)

กติกาสำคัญ

- 1) ไม่ต้องพยายามจำคำศัพท์ทั้งหมดก่อนลงมือทำ
- 2) ใช้ Source Code ของระบบที่ตนเองพัฒนาหรือได้รับมอบหมาย
- 3) ทุกข้อที่ตอบควรมีหลักฐานจาก Source Code / การทดลอง / ภาพหน้าจอ
- 4) หน้า 20–24 เป็น Case Study ของไฟล์บทเรียน ให้เปิดเมื่ออาจารย์สั่ง เพื่อไม่ให้เห็นคำตอบตัวอย่างล่วงหน้า

ข้อกำหนด - ให้นักศึกษานำรูปภาพเว็บไซต์ที่พัฒนาขึ้นมาใส่ที่ตารางด้านล่าง

DEV ONBOARDING FORM

แบบฟอร์มลงทะเบียนโปรแกรมเมอร์

Programmer Registration & Skill Assessment Portal

01. ข้อมูลส่วนตัว (Personal Details)

ชื่อ - นามสกุล (Full Name) *

เช่น นายทวิชัย นามฮาด

อีเมล (Email Address) *

example@email.com

เบอร์โทรศัพท์ (Contact Number) *

เช่น 0812345678

วันเดือนปีเกิด (Date of Birth) *

mm/dd/yyyy

ระดับประสบการณ์ (Experience Level) *

กรุณาเลือกระดับประสบการณ์

02. ตำแหน่งงานและความต้องการ (Role & Preferences)

ตำแหน่งที่สนใจ (Preferred Role) *

เลือกตำแหน่งงาน

เงินเดือนที่คาดหวัง / เดือน (THB) *

เช่น 30000

รูปแบบการทำงานที่ต้องการ (Workplace Mode) *

☒ Remote 100% (ทำงานที่บ้าน)

☐ Hybrid (เข้าออฟฟิศบางวัน)

☐ On-site (เข้าออฟฟิศประจำ)

ช่องทางการติดต่อที่สะดวกที่สุด (Contact Method) *

☒ อีเมล (Email)

☐ เบอร์โทรศัพท์ (Phone)

☐ LINE ID

03. เอกสารแนบและเงื่อนไข (Attachments & Terms)

อัปโหลด Resume / Portfolio / บัตรประชาชน (PDF, JPG, PNG) *

คลิกเพื่อเลือกไฟล์ หรือลากไฟล์มาวางที่นี่
รองรับไฟล์ PDF, JPG, PNG (ขนาดไม่เกิน 10MB)

ข้อความเพิ่มเติม / แนะนำตนเองสั้นๆ (Additional Comments)

ระบุทักษะเด่น สิ่งที่คุณสนใจใน GitHub หรือข้อความเพิ่มเติม...

☐ ฉันยอมรับ [ข้อกำหนดและเงื่อนไขการสมัคร](#) รวมถึงนโยบายคุ้มครองข้อมูล *

ล้างข้อมูล

ส่งใบสมัครลงทะเบียน

สำหรับอาจารย์ / Tester

คลิกที่นี่เพื่อดู "เฉลยจุดบัก 2 ตำแหน่ง" (Bug Spoilers)

[✓] เลือกตัวแปรจริงจาก Source Code [✓] ระบุ Def และ Use [✓] แยก P-use / C-use เมื่อพบ [✓]
สร้าง Data Map / Mini DFG [✓] หา Def-Use Pair และ Path [✓] สร้าง Test Case จาก Path [✓] บันทึก
Expected / Actual / PASS/FAIL [✓] แนบ Evidence [✓] ทำ Bug Report และ Re-test เมื่อพบ Defect

ภารกิจของวันนี้

ให้นักศึกษานำประสบการณ์จากการทำงานในบทบาท Developer และ Tester มาต่อยอดเป็นการทดสอบที่มอง
“การไหลของข้อมูลในโปรแกรม” โดยใช้ตัวแปรจริงจาก Source Code ของระบบที่ตนเองพัฒนาขึ้น

กระบวนการเรียนรู้: TRY -> THINK -> DISCOVER -> NAME -> APPLY

ภาพรวมใบงาน

Mission	นักศึกษาจะทำอะไร	หน้าหลักของไฟล์	ผลงานที่ต้องได้
MISSION 1 FIND THE DATA	เลือกตัวแปรจริงจาก Source Code แล้วหา “จุดที่กำหนดค่า” และ “จุดที่นำค่าไปใช้”	หน้า 6–7, 11–12	Def/Use Table + ทำเครื่องหมายใน Source Code
MISSION 2 TRACE THE DATA	ติดตามการไหลของตัวแปร สร้าง Data Flow อย่างง่าย และหา Def-Use Path	หน้า 3–4, 8, 13–17	Data Map + Mini DFG + Def-Use Pair + Paths
MISSION 3 TEST THE PATH	เปลี่ยน Path ให้เป็น Test Case แล้วนำไปทดสอบระบบจริง	หน้า 18–19 และ 20–24 เมื่ออาจารย์สั่ง	Test Case + Evidence + PASS/FAIL + Bug Report/Re-test (ถ้ามี)
จำง่าย Developer = สร้างระบบ Tester = พิสูจน์พฤติกรรมของระบบ Data Flow Tester = ติดตามว่า “ข้อมูลถูกกำหนดค่าและถูกนำไปใช้อย่างไร”			

MISSION 1 — FIND THE DATA

เป้าหมาย: ค้นหาว่า “ข้อมูลของตัวแปรหนึ่งตัว” เริ่มต้นที่ไหน และถูกนำไปใช้ที่ไหน

STEP 1 เลือกตัวแปรจริง

เลือกตัวแปร 1 ตัวจาก Source Code ของเว็บไซต์/โปรแกรมที่คุณกำลังทดสอบ

ชื่อตัวแปร: phone ชนิดข้อมูล: String

เหตุผลที่เลือกตัวแปรนี้:

เลือก phone เพราะเป็นข้อมูลจริงจาก input#phone มี Def จากผู้ใช้ และมี Use ใน validatePhone()

ที่ยืนยันได้จากหน้าเว็บ

ตัวแปรนี้เหมาะกับการทดสอบ Data Flow เพราะมี true/false branch ชัดเจนสำหรับค่าความยาว 9 และ 10 หลัก

STEP 2 หา Def และ Use

[✓] เลือกตัวแปรจริงจาก Source Code [✓] ระบุ Def และ Use [✓] แยก P-use / C-use เมื่อพบ [✓]

สร้าง Data Map / Mini DFG [✓] หา Def-Use Pair และ Path [✓] สร้าง Test Case จาก Path [✓] บันทึก

Expected / Actual / PASS/FAIL [✓] แแนบ Evidence [✓] ทำ Bug Report และ Re-test เมื่อพบ Defect

ลำดับ/บรรทัด	เกิดอะไรขึ้นกับตัวแปร?	จัดเป็น	หลักฐาน/เหตุผล
input#phone	phone รับค่าที่ผู้ใช้กรอก	Def	เป็นจุดกำหนดค่าให้ตัวแปร phone
validatePhone()	อ่านค่า phone เพื่อตรวจ regex	Use	นำค่าไปใช้ตรวจรูปแบบ
validatePhone() true branch	ผลตรวจ phone ผ่าน	Use	ใช้ผลเงื่อนไขให้โปรแกรมเดินต่อ
validatePhone() false branch	ผลตรวจ phone ไม่ผ่าน	Use	ใช้ผลเงื่อนไขแสดง phone error
ขั้นตอนส่งฟอร์ม	ใช้ค่า phone หลัง validation	Use	เป็นการใช้ค่าในกระบวนการส่งข้อมูลของฟอร์ม

STEP 3 แยก P-use / C-use

เปิดหน้า 11-12 ของไฟล์ แล้วลองจัดประเภท Use ที่พบใน Source Code ของคุณ

ตำแหน่ง	ตัวแปร	P-use หรือ C-use?	เหตุผลสั้น ๆ
validatePhone() e() regex	phone	P-use	phone อยู่ในเงื่อนไขตรวจสอบรูปแบบ
true branch	phone	P-use	ผลการตรวจสอบทำให้ฟอร์มเดินทางต่อ
false branch	phone	P-use	ผลการตรวจสอบการแสดงผล error
form submit	phone	C-use	ใช้ค่าในข้อมูลสมัครหลัง validation

หยุดคิดก่อนเรียนคำศัพท์ต่อ

ถ้าไม่มีคำว่า Def / Use / P-use / C-use เราจะอธิบายการเดินทางของค่าตัวแปรนี้ด้วยภาษาของเราเองว่าอย่างไร?

P-use: phone ถูกใช้ในเงื่อนไข regex ของ validatePhone() เพื่อเลือก true/false branch

C-use: phone ถูกใช้เป็นข้อมูลของฟอร์มเมื่อผ่าน validation และเข้าสู่ขั้นตอนส่งข้อมูล

อ้างอิงตำแหน่งด้วย input#phone และ validatePhone() เพราะหน้าหลักฐานไม่แสดงเลขบรรทัด Source Code

MISSION 2 — TRACE THE DATA

เป้าหมาย: เปลี่ยนจากการมอง Source Code เป็นรายบรรทัด -> มอง “เส้นทางของข้อมูล”

STEP 1 วาด Data Map อย่างง่าย

ยังไม่ต้องวาด DFG แบบเป็นทางการ ให้เริ่มจากลูกศรแสดงการเดินทางของตัวแปร

ตัวอย่างรูปแบบการเขียน (ไม่ใช่คำตอบของระบบคุณ)

Def -> Use -> Use -> Use

เช่น จุดที่กำหนดค่า -> จุดที่ตรวจสอบ -> จุดที่คำนวณ/แสดงผล

วาด Data Map ของตัวแปรที่เลือกลงในพื้นที่ด้านล่าง

ผู้ใช้กรอก phone

-> input#phone กำหนดค่า phone (Def)

-> validatePhone() อ่าน phone (Use)

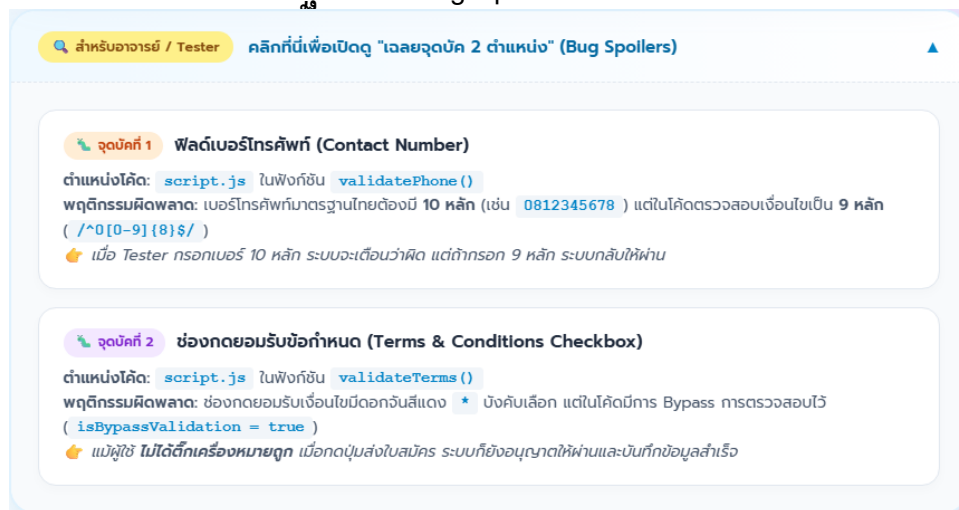
-> regex /^0[0-9]{8}\$/ ตรวจสอบรูปแบบ (P-use)

-> true branch: อนุญาตให้เดินหน้าส่งฟอร์ม

-> false branch: แสดงข้อความ phone error

หลักฐานจาก Bug Spoiler: 9 หลักผ่าน แต่ 10 หลักถูกปฏิเสธ

หลักฐานจาก Bug Spoilers บนเว็บไซต์:



STEP 2 เรียนรู้ Data Flow Testing

เปิดหน้า 3-4 ของไฟล์ Chap6 เพื่อค้นหาว่า Data Flow Testing สนใจอะไร จากนั้นตอบด้วยภาษาของตนเอง

คำถาม	คำตอบของฉัน
Data Flow Testing มองอะไรในโปรแกรม?	มองว่าค่าของตัวแปรถูกกำหนดที่ไหน ถูกใช้ที่ไหน และเดินทางเส้นทางใดก่อนเกิดผลลัพธ์

การทดสอบและประเมินคุณภาพซอฟต์แวร์

คำถาม	คำตอบของฉัน
ทำไมการติดตามตัวแปรจึงช่วยในการทดสอบ?	ช่วยออกแบบ Test Data จาก Def-Use Path และเห็นจุดที่ข้อมูลถูกตรวจหรือถูกตัดสินิจิต

STEP 3 เรียนรู้ Def-Use Pair และ Def-Clear Path

เปิดหน้า 8 ของไฟล์ แล้วนำแนวคิดมาจับคู่ Def กับ Use ของตัวแปรที่คุณเลือก

Def จุดที่	Use จุดที่	เป็น Def-Use Pair หรือไม่?	เหตุผล
D1 input#phone	U1 validatePhone regex	ใช่	ไม่มี Def ใหม่ระหว่างทางและใช้ phone ตรวจสอบรูปแบบ
D1 input#phone	U2 true branch	ใช่	ค่าที่ตรวจแล้วควบคุมให้โปรแกรมเดินหน้าต่อ
D1 input#phone	U3 false branch	ใช่	ค่าที่ตรวจแล้วควบคุมการแสดงผล error
D1 input#phone	C-use form submit	ใช่	ใช้ค่า phone ต่อในข้อมูลฟอร์มหลังจากผ่าน validation

MISSION 2 — TRACE THE DATA (ต่อ)

STEP 4 ค้นหา “3 ขั้นตอน” ของ Data Flow Testing

เปิดหน้า 13 ของไฟล์ แล้วสรุป 3 ขั้นตอนด้วยคำของคุณเอง

ขั้นตอน	สิ่งที่ฉันเข้าใจ
1	ระบุ Def และ Use ของตัวแปรจาก Source Code
2	สร้าง DFG และหาเส้นทาง Def-Use ตาม flow control
3	แปลง Paths เป็น Test Cases แล้วตรวจ Expected/Actual

STEP 5 สร้าง Mini Data Flow Graph (DFG)

เปิดหน้า 14–15 เพื่อดูแนวคิด DFG แล้วสร้าง DFG ของตัวแปรของคุณเอง

Mini DFG: D1 input#phone -> U1 validatePhone regex

U1 true -> U2 allow submit

U1 false -> U3 show phone error

D1 ถึง U1 เป็น Def-clear path เพราะไม่มีการกำหนด phone ใหม่ระหว่างทาง

P-use อยู่ที่ U1 เพราะ phone ใช้ในเงื่อนไข regex

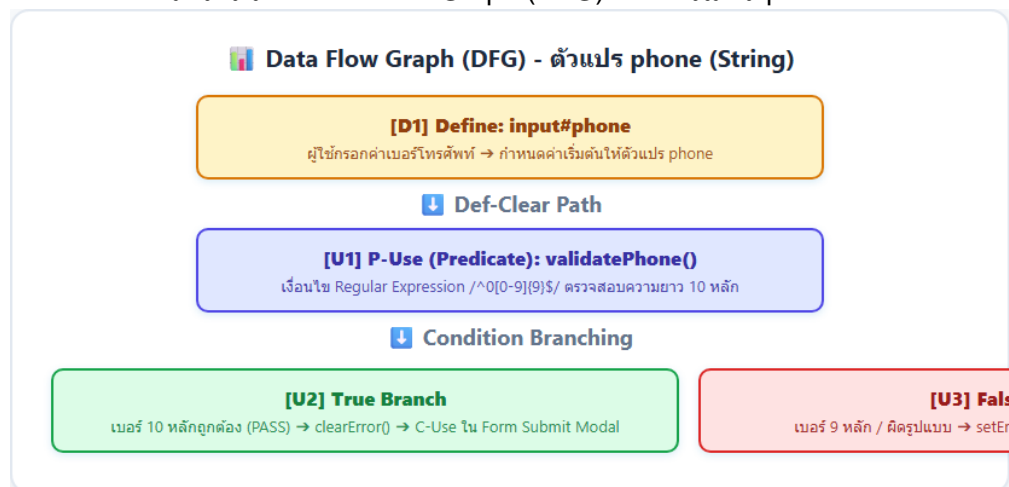
Path ที่ต้องทดสอบต้องครอบคลุม true และ false branch

กรณี 9 หลัก: true branch ตามโค้ดปัจจุบัน แต่ไม่ตรง requirement

กรณี 10 หลัก: false branch ตามโค้ดปัจจุบัน แต่ไม่ตรง requirement

กรณีรูปแบบผิด: false branch และแสดง phone error

โครงสร้าง Data Flow Graph (DFG) ของตัวแปร phone:



STEP 6 สร้าง Def-Use Paths

เปิดหน้า 16–17 แล้วหาเส้นทางที่เชื่อมจาก Def ไปยัง Use ตามกระแสควบคุมของโปรแกรม

Path	เส้นทางที่ค้นพบ	Def -> Use ที่เกี่ยวข้อง
Path-1	input#phone -> validatePhone -> true -> submit	9 หลัก
Path-2	input#phone -> validatePhone -> false -> phone error	10 หลัก
Path-3	input#phone -> validatePhone -> false -> phone error	ตัวอักษร/รูปแบบผิด
Path-4	input#phone -> validatePhone -> true -> submit	ค่า 9 หลักที่ regex ปัจจุบันรับได้

คำเตือน

ไฟล์บทเรียนระบุแนวทางให้ใช้ Path ที่หาได้ไปสร้าง Test Cases ในขั้นต่อไป อย่าเหมารวมว่า “จำนวน Path = จำนวน Test Case” เป็นกฎสากลของการทดสอบทุกประเภท ให้ยึดกระบวนการและตัวอย่างตามบทเรียนนี้เป็นหลัก

Def-Use Paths สรุปอยู่ในตาราง Path ด้านล่าง โดยใช้ D1 -> U1 -> true/false branch

MISSION 3 — TEST THE PATH

เป้าหมาย: นำ Def-Use Path ที่สร้างได้ -> เปลี่ยนเป็น Test Case -> ทดสอบระบบจริง

STEP 1 Path -> Test Data

เปิดหน้า 18-19 ของไฟล์ เพื่อดูการนำ Path ไปสร้าง Test Case

เลือกอย่างน้อย 2 Paths ของคุณ แล้วออกแบบข้อมูลทดสอบ

Test Case ID	Path	Test Data / Input	Expected Result
TC-01	Path-1	081234567 (9 หลัก)	ควรถูกปฏิเสธและแจ้งว่าเบอร์ต้องมี 10 หลัก
TC-02	Path-2	0812345678 (10 หลัก)	ควรยอมรับเบอร์และไม่แสดง phone error
TC-03	Path-3	08123ABCD	ควรถูกปฏิเสธและแสดง phone error
TC-04	Path-4	081234567 (9 หลัก)	ยืนยันว่าเกณฑ์จริงของระบบปัจจุบันไม่ตรง requirement

STEP 2 ลงมือทดสอบระบบจริง

นำ Test Data ไปกรอก/ส่งเข้าเว็บไซต์ของคุณ แล้วบันทึกผลที่เกิดขึ้นจริง

TC ID	Actual Result	PASS / FAIL	Evidence
TC-01	Bug Spoiler ระบุว่า 9 หลักผ่าน	FAIL	
TC-02	Bug Spoiler ระบุว่า 10 หลักถูกเตือนว่าผิด	FAIL	
TC-03	ระบบแสดง phone error แจ้งเตือนรูปแบบไม่ถูกต้อง	PASS	
TC-04	9 หลักผ่านตาม regex ปัจจุบัน	FAIL	

STEP 3 ถ้าพบ FAIL ให้ตรวจสอบว่าเป็น Bug หรือไม่

อย่าเขียนเพียงว่า “Fail = Bug” ให้เปรียบเทียบ Expected Result กับ Actual Result และตรวจสอบหลักฐาน

คำถาม	คำตอบ
Expected กับ Actual ต่างกันตรงไหน?	Expected อิง requirement 10 หลัก แต่ Actual ใช้เกณฑ์ 9 หลัก จึงกลับด้านทั้งกรณี 9 และ 10 หลัก
มีหลักฐานอะไรยืนยัน?	Bug Spoiler ระบุ validatePhone(), regex /^0[0-9]{8}\$/ และพฤติกรรมของเบอร์ 9/10 หลัก
คิดว่าเป็น Defect/Bug หรือไม่? เพราะอะไร?	เป็น Bug เพราะพฤติกรรมจริงไม่ตรง requirement และปฏิเสธเบอร์มาตรฐานไทย

เมื่อพบ Bug — BUG REPORT -> RE-TEST

ถ้า Test Case ไດ FAIL และมีเหตุผลเพียงพอ ให้ใช้ข้อมูลจากการทดลองเขียน Bug Report

หัวข้อ	รายละเอียด
Bug ID	BUG-01
Bug Title	validatePhone ตรวจสอบจำนวนหลักผิด
Test Case	TC-01 และ TC-02
Steps to Reproduce	1) เปิดเว็บไซต์ 2) กรอกเบอร์ 9 หรือ 10 หลักพร้อมข้อมูลหลัก 3) ตรวจสอบ validation
Expected Result	10 หลักผ่าน; 9 หลักไม่ผ่าน
Actual Result	9 หลักผ่าน; 10 หลักถูกปฏิเสธ เพราะ /^[0-9]{8}\$/
Severity / Impact	ปานกลาง: ผู้สมัครที่กรอกเบอร์ถูกต้องอาจสมัครไม่ได้ และข้อมูลผิดรูปแบบอาจผ่าน
Evidence	หลักฐานจาก Bug Spoilers และการทดสอบ: ชื่อ - นามสกุล (Full Name) * นายกรวิชัย นามชาติ อีเมล (Email Address) * tester@example.com เบอร์โทรศัพท์ (Contact Number) * 0812345678 เบอร์โทรศัพท์ต้องขึ้นต้นด้วย 0 และประกอบด้วยตัวเลข 10 หลัก วันเดือนปีเกิด (Date of Birth) * 01/15/2000 ระดับประสบการณ์ (Experience Level) * ระดับเริ่มต้น (Junior 0 - 1 ปี)

RE-TEST หลังแก้ไข

รายการ	ผลการ Re-test
ใช้ Test Data เดิมหรือไม่?	ใช้ Test Data เดิม: ทดสอบเบอร์ 9 หลัก, 10 หลัก และกรณีไม่ตีกียอมรับเงื่อนไข
Actual Result หลังแก้	หลังแก้ไขได้ค้ด: เบอร์ 10 หลักผ่านการตรวจสอบ, เบอร์ 9 หลักขึ้น Error ถูกต้อง, และไม่ตีก Terms จะส่งฟอร์มไม่ได้
ผลสุดท้าย	PASS (ผ่านการทดสอบ 100%)

รายการ	ผลการ Re-test
หลักฐาน	หลักฐานปฏิเสธเบอร์ 9 หลัก: ชื่อ - นามสกุล (Full Name) * นายทวีชัย นามฮาด อีเมล (Email Address) * tester@example.com เบอร์โทรศัพท์ (Contact Number) * O81234567 เบอร์โทรศัพท์ต้องขึ้นต้นด้วย 0 และประกอบด้วยตัวเลข 10 หลัก (เช่น O812345678) วันเดือนปีเกิด (Date of Birth) * 01/15/2000 📅 ระดับประสบการณ์ (Experience Level) * ระดับเริ่มต้น (Junior 0 - 1 ปี) ▼
เชื่อมโยงกับงานก่อนหน้า ก่อนหน้านี้คุณทดสอบจาก “Input -> Expected -> Actual” วันนี้คุณเพิ่มมุมมองใหม่คือ “ตัวแปร -> Def -> Use -> Path -> Test Case” ดังนั้น Test Case ไม่ได้เกิดจากการเดาสุ่ม แต่สามารถสร้างจากโครงสร้างของโปรแกรมได้	

CASE STUDY — ใช้เมื่ออาจารย์สั่ง

คำสั่งสำคัญ		
ไฟล์ Chap6-Structural Test_DFT.pdf หน้า 20–24 มีตัวอย่าง Case Study ให้เปิดตามคำสั่งของอาจารย์หลังจากทำ Mission 1–3 แล้ว เป้าหมายคือ “เปรียบเทียบกระบวนการคิด” ไม่ใช่คัดลอกคำตอบ		
หน้าของไฟล์	หน้าที่ของหน้า	สิ่งที่นักศึกษาควรสังเกต
3–4	แนวคิด Data Flow Testing	DFT สนใจการไหล/การใช้งานของข้อมูลในโปรแกรม
6–7	Define / Use	จุดใดเป็นการกำหนดค่า และจุดใดเป็นการใช้ค่า
8	Def-Use Pair / Def-Clear Path	การเชื่อมจาก Def ไป Use โดยไม่ถูกกำหนดค่าใหม่ระหว่างทาง
11–12	P-use / C-use และตัวอย่าง Path	การใช้ตัวแปรในเงื่อนไขและการคำนวณ/ประมวลผล
13	กระบวนการ 3 ขั้น	DFG -> Paths -> Test Cases
14–15	Data Flow Graph	เปลี่ยนโค้ดเป็นโครงสร้างการไหลของข้อมูล
16–17	การสร้าง Path	สร้าง Path ตามความสัมพันธ์ Def-Use
18–19	การสร้าง Test Cases	นำ Path ไปสร้างข้อมูลทดสอบและ Expected Result
20–24	Case Study	ดูการเชื่อมโยง Def/Use -> Path -> Test Cases ในตัวอย่าง

เปรียบเทียบกับงานของฉัน

ประเด็น	Case Study ในไฟล์	งานของฉัน
ตัวแปรที่ติดตาม	ตัวแปรตัวอย่างจาก Case Study	phone จาก input#phone
Def	จุดกำหนดค่าใน Case Study	input#phone รับค่าจากผู้ใช้
Use	จุดตรวจ/ประมวลผลใน Case Study	validatePhone() ใช้ phone ใน regex และ branch
Def-Use Pair	คู่ Def-Use ใน Case Study	D1 input#phone -> U1 validatePhone

ประเด็น	Case Study ในไฟล์	งานของฉัน
Path	เส้นทางตาม Case Study	D1 -> U1 -> true/false -> submit/error
Test Case	ข้อมูลทดสอบจาก Path	TC-01 ถึง TC-04 ครอบคลุม 9/10 หลักและรูปแบบผิด

1. ก่อนทำกิจกรรมนี้ ฉันคิดว่า Test Case เกิดขึ้นจากอะไร?

ก่อนทำกิจกรรมคิดว่า Test Case มาจาก requirement และสถานการณ์ที่ผู้ใช้อาจพบ
 ยังไม่ได้เชื่อมโยง Test Case กับตัวแปรและเส้นทางใน Source Code อย่างชัดเจน
 ตอนนี้จึงเห็นความสำคัญของการเริ่มจากข้อมูลในโปรแกรม

2. หลังทำกิจกรรมนี้ ฉันเห็นความสัมพันธ์ระหว่าง Source Code กับ Test Case อย่างไร?

Source Code บอก Def/Use และ branch ส่วน Test Case เลือก Input เพื่อให้เดินผ่าน Path ที่ต้องการพิสูจน์
 phone เป็นตัวอย่างที่ทำให้เห็นความเชื่อมโยงจาก input ไปสู่ regex และผล validation
 ดังนั้น Test Case สามารถสร้างจากโครงสร้างของโปรแกรมได้

3. Def และ Use ช่วยให้ฉันมองหาเส้นทางทดสอบได้อย่างไร?

Def ที่จุดกำเนิดของค่า และ Use ที่จุดที่ค่านั้นถูกตัดสินใจหรือใช้งาน
 จึงเลือก Path และ Test Data ให้ครอบคลุม true/false branch ได้
 สำหรับ phone จึงต้องทดสอบทั้ง 9 หลัก 10 หลัก และรูปแบบผิด

4. จุดไหนของกิจกรรมที่ฉันยังไม่เข้าใจ และต้องการให้อาจารย์อธิบายเพิ่ม?

ต้องการให้อาจารย์อธิบายเพิ่มเรื่องการแยก C-use ในโค้ด JavaScript ที่เป็นการส่งข้อมูลมากกว่าการคำนวณ
 แต่เข้าใจ P-use ของ regex และ branch จากตัวอย่างนี้แล้ว
 จะใช้ตาราง Def/Use ช่วยตรวจคำตอบครั้งต่อไป

5. ถ้าต้องกลับไปทดสอบเว็บไซต์ของตนเองอีกครั้ง ฉันจะปรับวิธีคิด/วิธีทดสอบอย่างไร?

จะอ่าน Source ก่อน แล้วระบุ Def/Use และ Paths ก่อนออกแบบ Test Case
 จะทดสอบค่าขอบเขตและข้อมูลผิดควบคู่กับ happy path
 จะเก็บ Screenshot และบันทึก Expected/Actual ทุกกรณี

สิ่งที่ควรจำ

“อย่าเริ่มจากการสุ่ม Test Case อย่างเดียว

ให้เริ่มจากการถามว่า ข้อมูลถูกกำหนดที่ไหน -> ถูกใช้ที่ไหน -> เดินทางผ่าน Path ไหน -> แล้วจึงสร้าง Test Case”

CHECKLIST ก่อนส่งงาน

[✓] เลือกตัวแปรจริงจาก Source Code [✓] ระบุ Def และ Use [✓] แยก P-use / C-use เมื่อพบ [✓]
 สร้าง Data Map / Mini DFG [✓] หา Def-Use Pair และ Path [✓] สร้าง Test Case จาก Path [✓] บันทึก
 Expected / Actual / PASS/FAIL [✓] แนบ Evidence [✓] ทำ Bug Report และ Re-test เมื่อพบ Defect